Large-scale database – Notes

Francesco Passino

Contents

1 Key-value Databases **2**
- 1.1 simplicity . . . 2
- 1.2 speed . . . 2
- 1.3 Scalability . . . 2

2 Lesson bho? — 13/10/2026 **2**
- 2.1 Document Database Introduction . . . 2
  - 2.1.1 XML documentes . . . 3
  - 2.1.2 JSON Documents . . . 3
  - 2.1.3 Schema-less vs Schema . . . 3
- 2.2 Data modelling and partitioning . . . 3
  - 2.2.1 Document Embedding . . . 3
  - 2.2.2 Document Linking . . . 4

3 lesson x — 14/10/2026 **4**
- 3.1 One to many reletionship . . . 4
- 3.2 Many to many reletionship . . . 4

1 Key-value Databases

In key-value database:

- Data are persistently stored
- There is no need for table, we do not need group of related attributes.
- The unique requirement is that each value has an unique identifier (key).
- Keys must be uniques within the namespace define inside each specific bucket.

Usually key-value database are used when the main goal si ease of storage and retrieval and not organizing complex data structure

1.1 simplicity

1.2 speed

One essential feature of Key-value database is speed. This is beacuse most of the key-value database are loaded in memory and by using cache we can speed up read and write operation.

1.3 Scalability

Definition 1.1.
the scalability (horizontal) is the capability to add or remove servers from a cluster of servers as needed to accommodate the load on the system

`Master-slave architecture`:

- the master is the server that receive all the write operation.
- the slaves only respond to read request and they do not communicate among them.

Pros: very simple architecture with no need to coordination in write operation Problems: If the master fails $\implies$ by by.

We can complicate this architecture allowing the slave to communicate among them. The slaves ask to the other slaves if someone has received response from the master, if not they can choose someone to be promoted as a master.

`Masterless architecture` There is no master node that has the master copy of the data but each node is in charge of helping his neighbors. This architecture implies that the nodes are more all less the same , from a software and hardware point of view, they can all compute reade and write operation and are able to handle replication. Each time there is a write operation that update a key on one of the server it propagate to its two neighbors and to the server two nodes ahead(in case we want 4 replica, we can choose how many). After this process the updated key is store in 4 different nodes to avoid SPF.

2 Lesson bho? — 13/10/2026

2.1 Document Database Introduction

In document database the single piece of information is a structured document, usually the format chosen is `xml` or `json`. Structured documents provide `high flexibility` and the possibility to handle complex data structure. Other important features are that Document database are `schema-less` and that they allow complex operations (queries and filtering) and `ACID transactions`.

2.1.1 XML documentes

XML stand for `eXtensibke Markup Language`, a markup language specifies the structure and the content of a document and includes `tags` to provide extra information. With XML we can store any form of information and is typically used in website to separate data and format or as a standard for some document type(docx, pptx). Some advantages of XML are:

- XML is text based (easily transmitted).
- One XML document can be displayed differently in different media.
- XML documente can be modularized.

Some problems and drawbacks of XML are:

- tags are verbose and repetitious (we need more storage space)
- XML documents are wasteful of space and computationally expensive.
- JSON is better

2.1.2 JSON Documents

JSON stand for `JavaScript Object Notation` and it's used to format data, a json document has been preferred to an XML one for `data intercharging` on the internet. In JSON an object is an unordered set of `name/value` pairs, the paires are enclosed within braces({}), the name and the value are separeted by a colon(:) and pairs are separeted by commas. The values can be many object: another json object, string, array(enclosed in brackets[]), number, boolean adn Null.

We will study json based database where the data is stored in JSON format of course, document may bee stored in collection and there is not a predefined scheme.

2.1.3 Schema-less vs Schema

A schema is a specification that describes teh structure of an object, such as table. This specifications need to be define before executing any code and we can't change the structure of the objects after. In document database we do not need to define any structure and developers can create collections by simply inserting them into the database. In schema-less database we have high flexibility in handling the structure of object but the DBMS may be not allowed to enforce rules based on the structure of the data, is more difficult to perform integrity check.

2.2 Data modelling and partitioning

We will see how to model document database given some requirements.

2.2.1 Document Embedding

Using document database we could simulate a relational database using collection but DON'T DO IT, instead we will use nested documents to represent relationships among entities. Usually we want to reduce the number of collection used. Using nested documents (`Document Embedding`) we can retrieve both entities in a single operation, we need to be careful because we can introduce duplicate across multiple documents and in complex design this could lead to issues or inconsistencies in case of changes. We need to be care on the embedding we do and choose carefully depending on the implementation.

2.2.2 Document Linking

For the too many relations we can also used another approach, Document linking. In this approach we can define a collection of entities and inside a list of ID refearing to another entities collection. DO NOT USE IN THE PROJECT except for some specific cases.

3 lesson x — 14/10/2026

3.1 One to many reletionship

We will consider as example the customer entity which may gave assosueted a list of address entities. Cusstomer have an ID, a name and string "*person_or_business*". In a one to many relationship the primary documente is teh `one` (customer) and the `many` entities (address) are represented as an array of embedded dovuments. We have some drawback using this approach: we could have consistency problem if there are the same address for diffrent customers; if a customer have a lot of address the dimension increase too much,(dimension could lead to inefficienty problem) we need to choose a max dimension; we could have problem with mutable document or dinamically dimension. For mutable document is better the linking documente approach if we have limits in document domension and we don't need to access the "too many" very often.

3.2 Many to many reletionship

As example we can consider two entities: students and courses. We can use two collection and in both add an array of the IDs of the other one (we need to be careful on integrity, no one checks exept us).

esempio delle slide (anche non completo ma va bene una istanza per entità)

Another solution is to implement a partial document embedding: partial becauso we do not include all the values of the document but just the ones we want. For exemple in the student collection we can embed the partial documente of the course with just ID and title of the course.

metti un esempio della collection student e uno dei course

The third approach is to mix embedding and linking

metti un esempio della collection student e uno dei corsi